

# Normes bàsiques d'escriptura de bon codi en Java

Aquestes normes serveixen perquè el codi no només funcioni, sinó que també sigui clar, ordenat i fàcil d'entendre.

Un programa ben escrit:

és més fàcil de corregir,

és més fàcil de mantenir,

## 1. Com s'han d'anomenar les classes

Les classes s'han d'escriure amb PascalCase.

Què és PascalCase?

Cada paraula comença amb majúscula i no hi ha espais ni guions baixos.

Exemples: Persona, GestorFitxers, TaulerEscacs, Partida8Reines

Incorrecte: persona, gestor\_fitxers, tauler\_escacs, partida8reines

```
public class Alumne { }  
public class Cotxe { }  
public class Matriu { }  
public class GestorUsuaris { }
```

## 2. Variables i atributs

S'escriuen en camelCase.

Què és camelCase? Primera paraula en minúscula i la resta amb majúscula.

Exemples: nom, edat, notaFinal, nombreAlumnes

Incorrecte: Nom, nota\_final, nombre\_alumnes, x, a

```
private String nomAlumne;  
private int edat;  
private double notaMitjana;  
private boolean partidaAcabada;
```

Recomanació important: els noms han de ser descriptius.

```
// Correcte  
int nombreReines;  
double notaMitjana;  
String nomFitxer;
```

```
// Incorrecte
int n;
double x;
String s;
```

Només es permeten noms curts en casos concrets:

```
for (int i = 0; i < 8; i++) { }

for (int i = 0; i < files; i++) {
    for (int j = 0; j < columnes; j++) {
        ...
    }
}
```

També és recomanable comentar els atributs importants:

```
/**
 * Nom complet de l'alumne.
 */
private String nom;
```

### 3. Constants

S'escriuen en MAJÚSCULES\_AMB\_GUIONS i amb final.

```
public static final int MAX_ALUMNES = 30;
public static final int MIDA_TAULER = 8;
public static final double NOTA_MAXIMA = 10.0;
```

Incorrecte:

```
public static final int maxAlumnes = 30;
public static final int midaTauler = 8;
```

Les constants també s'han de comentar si no són evidents:

```
/**
 * Nombre màxim d'alumnes permesos al grup.
 */
public static final int MAX_ALUMNES = 30;
```

### 4. Mètodes

Han de ser accions i escrits en camelCase.

```
public void mostrarDades() { }
public double calcularArea() { }
public boolean esMovimentValid() { }
```

## 5. Comentaris

, Explicar el que no és evident per qui no ha escrit el codi i les particularitats que pugui haver-hi. Tant per les classes, atributs, constructors, mètodes, paràmetres, constants i codi.

Mala idea:

```
// sumam 1 a i
i = i + 1;
```

Bona idea:

```
// Reiniciem el comptador perquè començam una nova fila del tauler
comptador = 0;
```

## 6. Parametrització

Evitar valors màgics.

Malament:

```
if (edat >= 18) { }
```

Millor:

```
public static final int EDAT_MAJORIA = 18;
if (edat >= EDAT_MAJORIA) { }
```

## 7. Exemple complet

```
/**
 * Classe que representa un alumne amb nom, edat i nota final.
 */
public class Alumne {

    public static final double NOTA_MINIMA = 0.0;
    public static final double NOTA_MAXIMA = 10.0;

    private String nom;
    private int edat;
    private double notaFinal;

    public Alumne(String nom, int edat, double notaFinal) {
        this.nom = nom;
        this.edat = edat;
        this.notaFinal = notaFinal;
    }

    public String getNom() {
```

```
        return nom;
    }

    public void setNom(String nom) {
        this.nom = nom;
    }

    public boolean estaAprovat() {
        return notaFinal >= 5.0;
    }
}
```

## **7. Javadoc.**

En Java, es pot usar javadoc per a documentar els programes, és una eina que permet generar codi web amb tota la documentació de l'aplicació. Les IA poden ser especialment útils per aquesta tasca de documentació.